Project 1: Internal Employee Portal Audit (Altschool Case)

Organization: AltSchool Africa
Assessment Team: AltSchool Ethical Hacking Team
Assessment Type: Academic / Corporate-Style Penetration Test Report
Environment: PortSwigger Web Security Academy (Simulated Internal Portal)
Date: January 2026

1. Executive Summary

I set out to see how well the access controls work inside AltSchool Africa's simulated employee portal.

The main question: can users only see their own account info, or is there a loophole? Turns out, there's a serious gap. The portal requires a user input request parameter to determine which record to show. That means if you're logged in, you can just manipulate that parameter and peek at someone else's data. The app doesn't check if you own the account you're trying to access. This could lead to confidential data leaks across the organization.

2. Scope and Methodology

2.1 Scope

The employee account details feature of the portal was the focus of this test. An employee account was used, but only authentication and authorization controls were tested.

2.2 Methodology

We took a manual black-box approach as per OWASP web security testing guide. After logging in as a normal user, we noticed how the app functioned and intercepted the HTTP traffic with Burp Suite. We searched for parameters which would let users gain control and tried varying these. We were not there to visit another person's data.

3. Reconnaissance and Initial Observations

Target URL:
https://portswigger.net/web-security/access-control/lab-user-id-controlled-by-request-parameter-with-data-leakage-in-redirect

Authentication Context:
Authenticated as ***wiener:peter***

Observed Behavior:
After logging in, the app showed account details for the logged-in user as expected. Without logging in, access was blocked, so authentication does work but when I dug a little deeper, I realized there's barely any authorization between user accounts.

4. Vulnerability Identification

4.1 Vulnerability Description

Category: OWASP Top 10-A01 Broken Access Control

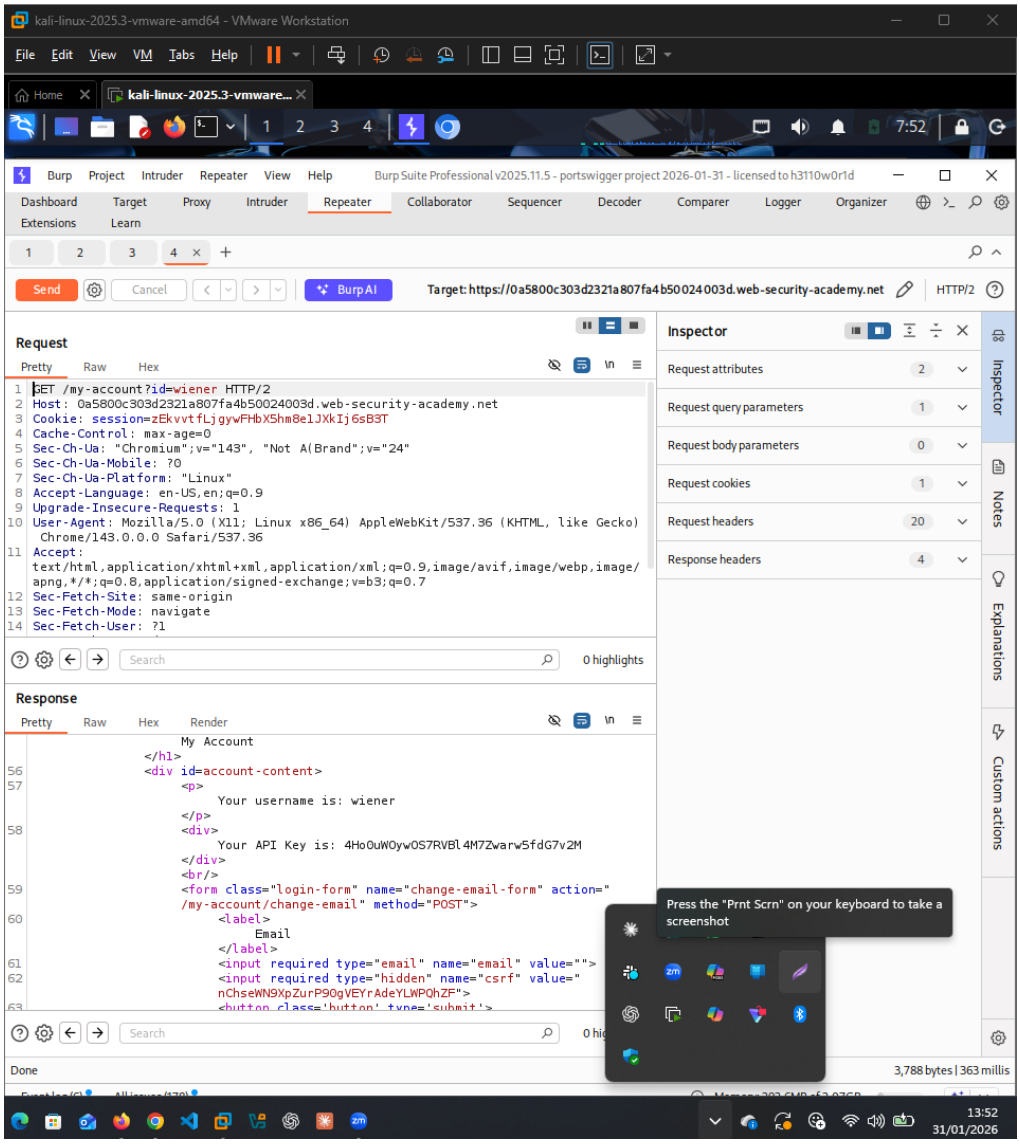
Type: User ID controlled by request parameter with data leakage in redirect

The app depends on an "id" parameter sent by the user to decide which account data to pull up but the server doesn't check if the account being requested belongs to the logged-in user.

4.2 Vulnerable Request

```
GET /my-account?id=wiener HTTP/2
```

The server validates this parameter blindly, any user who's logged in can change it and get another user's data.



5. Proof of Concept (PoC)

5.1 Exploitation Technique

The “id” parameter was changed from *weiner* to *carlos*:

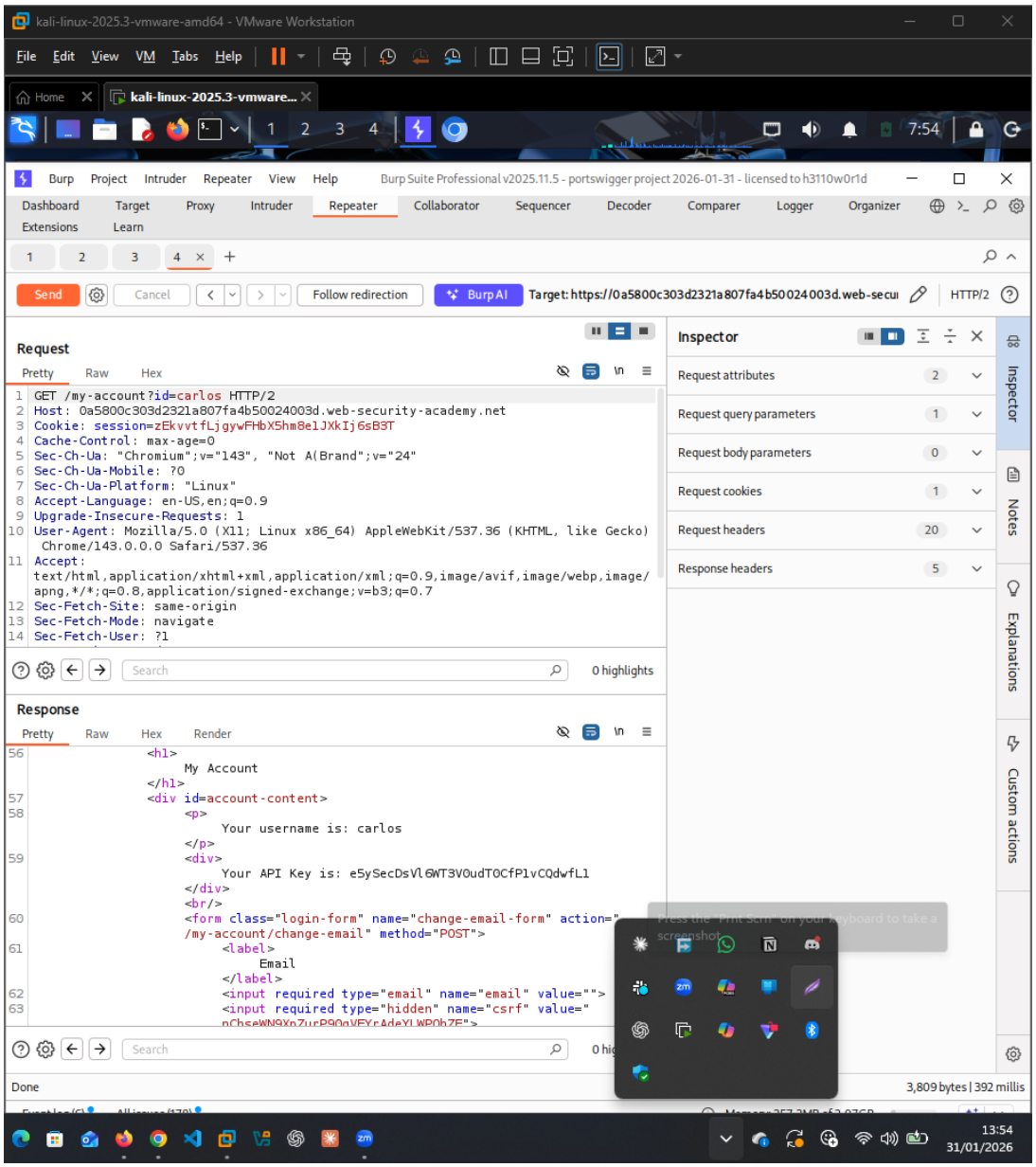
```
GET /my-account?id=carlos HTTP/2
```

5.2 Result

I got a *HTTP/2 302* redirect from the server. Even with the redirect, sensitive info attached to the other user showed up.

Exposed Data:

- API key belonging to another user account (This shows that the application does not perform object-level authorization checks before returning sensitive data).



(Screenshot: Burp Suite response showing API key leakage in redirect)

6. Impact Assessment

Any logged-in employee can snoop on other people's private info because of this vulnerability. That's a huge problem because anyone with access can get or manipulate sensitive information belonging to other users.

7. Use of AI Assistance and Transparency Statement

I wrote this report and used ChatGPT to help improve the structure of my report. The Lab, observations, and conclusions are mine, no result was generated by AI

7.1 Prompt Used

"Rewrite my penetration testing report to sound more academic and corporate. Improve clarity and flow, reduce informal language, and polish the final document while preserving all technical details. I will provide all findings, scope, methodology, and evidence."

8. References

• OWASP Foundation. OWASP Top 10 Web Application Security Risks - A01: Broken Access Control.

• PortSwigger Web Security Academy. Access Control Vulnerabilities.

• OpenAI. ChatGPT